Rotational Packet Protocol (RPP): A Semantic Addressing Architecture for Consent-Aware Memory Systems

Alexander Liam Lennon
Independent Researcher
https://github.com/anywave/rpp-spec

March 2026
Version 2.0.0

Abstract

We present the Rotational Packet Protocol (RPP) v2.0, a novel two-layer addressing architecture that encodes semantic meaning, access consent, and lifecycle state directly into fixed-width addresses. The Semantic Interface Layer (v1.0) defines a 28-bit address `Shell[2] | Theta[9] | Phi[9] | Harmonic[8]` providing a human-meaningful developer API for storage tier, functional sector, grounding level, and routing mode. The Transport/Resonance Layer (v2.0) defines a 32-bit Ra-Canonical address `[`θ`:5]` `[`ϕ`:3]` `[h:3]` `[r:8]` `[reserved:13]` for substrate routing across Repitan sectors, RAC bands, and Omega tiers. Both layers share a toroidal address space $T^2 = S^1 \times S^1$ whose cyclic geometry makes rotation a natural and efficient encryption primitive. The Pong key derivation scheme produces ephemeral consent-field keys via golden-ratio modular arithmetic; Skyrmion winding numbers provide topological authentication that is structurally impossible to forge piecewise. The Ford Protocol implements five-phase Hold-Not-Drop substrate crossing for ConsciousnessStatePackets (CSPs) that cannot be retransmitted. Consent-field mesh routing uses gradient descent on the torus with no routing table: a single arithmetic comparison ($\phi_{\text{packet}} < \phi_{\text{min}}^{\text{node}}$) enforces ACCEPT, FORWARD, or BARRIER at every hop with no policy server. The same 28-bit address resolves identically over IPv4/UDP, LoRa, IPFS, and Hedera Hashgraph; only the byte envelope changes. Empirical evaluation on a 50-node simulated network with 1,000 random packets achieves 100% convergence, mean 0.99 hops, zero routing loops, with 40.8% of packets BARRIER'd at consent gates. Python reference performance: encode 662 ns (rformance: encode 662 ns (1.5 M/s), decode 521 ns (1.9 M/s), routing decision 3.4 μs (294 K/s). GDPR structural compliance is arithmetic: a phi shift makes the old address unroutable at every node simultaneously with no DELETE cascade. This paper is a defensive disclosure releasing all described mechanisms as open infrastructure under CC BY 4.0.

Keywords: semantic addressing, consent-aware systems, memory architecture, geometric addressing, toroidal geometry, rotational encryption, Ford Protocol, transport-modality agnostic, bridge architecture, open infrastructure, packet protocol, GDPR compliance

Defensive Publication Notice: This paper constitutes a defensive disclosure establishing prior art. The authors explicitly claim no patent rights and intend to prevent future patent enclosure of the described architecture. Released under CC BY 4.0.

Contents

1 Introduction **4**
1.1 Problem Statement 4

1.2 Contribution . . . 4
1.3 Explicit Non-Patent Statement . . . 5

2 The 28-Bit RPP Address (Semantic Interface Layer) 5
2.1 Bit Layout . . . 5
2.2 Field Definitions . . . 5
2.3 Encoding Algorithm . . . 5
2.4 Decoding Algorithm . . . 6
2.5 Reserved Bits . . . 6

3 Two-Layer Architecture 6
3.1 Overview . . . 6
3.2 Ra-Canonical Transport Layer (v2.0) . . . 6
3.3 Layer Translation . . . 7

4 Toroidal Geometry and Rotational Encryption 7
4.1 Why a Torus . . . 7
4.2 Toroidal State Vector . . . 8
4.3 Pong Rotational Encryption . . . 8
4.4 Skyrmion Winding Authentication . . . 9
4.5 Security Bounds . . . 9

5 Ford Protocol: Substrate Crossing 9
5.1 Hold Not Drop . . . 9
5.2 ConsciousnessStatePacket Structure . . . 10
5.3 Recovery Escalation . . . 10
5.4 Shell TTL in Liminal Timeout . . . 10

6 The Rotational Packet 10
6.1 Packet Structure . . . 10
6.2 Packet Fields . . . 11
6.3 Payload Types . . . 11

7 Semantic Interpretation 11
7.1 Shell (Radial Depth) . . . 11
7.2 Theta (Functional Sector) . . . 11
7.3 Phi (Grounding Level and Consent Threshold) . . . 11
7.4 Harmonic (Mode/Resolution) . . . 11

8 Consent-Field Mesh Routing 12
8.1 No Routing Table . . . 12
8.2 Routing Decision Arithmetic . . . 12
8.3 GDPR Structural Compliance . . . 13

9 Transport-Modality Agnosticism 14
9.1 Substrate Portability . . . 14
9.2 Fallback Chain . . . 14
9.3 PARL Hardware Targets . . . 14

10 Comparison to Existing Architectures **15**
10.1 CPU Virtual Memory (x86/ARM) . . . 15
10.2 GPU Memory (CUDA/Vulkan) . . . 15
10.3 Content-Addressable Memory (CAM) . . . 15
10.4 Object Storage (S3/GCS/ZFS) . . . 15

11 Prior Art Differentiation **15**
11.1 What RPP Does NOT Claim as Novel . . . 15
11.2 What RPP DOES Claim as Novel Synthesis . . . 16

12 Resolver Architecture **16**
12.1 Bridge Model . . . 16
12.2 Resolver Interface . . . 17

13 Consent and Coherence Gating **17**
13.1 Consent States . . . 17
13.2 Address-Level Gating . . . 17

14 Hardware Considerations **18**
14.1 Why 28 Bits . . . 18
14.2 Historical Precedent . . . 18
14.3 FPGA Implementation . . . 18

15 Empirical Validation **18**
15.1 Routing Convergence . . . 19
15.2 Performance Benchmarks . . . 19
15.3 GDPR Structural Compliance Demonstration . . . 19

16 Implementation Status **20**

17 Conclusion **20**

A Test Vectors **21**
A.1 Semantic Interface Layer (v1.0) Test Vectors . . . 21
A.2 Ra-Canonical Transport Layer (v2.0) Test Vector . . . 22

B Licensing **22**

1 Introduction

1.1 Problem Statement

Modern computing architectures treat memory addresses as opaque numerical identifiers:

$$\text{address} \rightarrow \text{location} \rightarrow \text{bytes} \tag{1}$$

This model, established in the von Neumann architecture and refined through decades of CPU design, optimizes for sequential access patterns, cache locality, hardware simplicity, and byte-level granularity. However, it provides no intrinsic mechanism for:

- Semantic classification of data
- Consent-based access control beyond binary ACLs
- Lifecycle management at the address level
- Functional routing without lookup tables
- Substrate-independent address portability
- Cryptographic authentication without external key stores

As systems increasingly deal with meaning (embeddings, knowledge graphs, AI-generated content, edge-deployed cognitive agents) rather than raw bytes, the gap between address semantics and data semantics creates architectural friction. Consent management in particular is typically an afterthought: GDPR compliance requires application-level DELETE cascades, privacy-preserving routing requires policy servers, and cross-substrate state transfer (e.g., moving an active computation from a cloud VM to an embedded microcontroller) has no standard mechanism.

1.2 Contribution

We propose RPP v2.0, a two-layer addressing model where:

1. **Address = Classification**: The 28-bit Semantic Interface address directly encodes what data means, not just where it resides.
2. **Access = Arithmetic**: Consent enforcement is a single integer comparison at each routing node, with no policy server or coordination.
3. **Routing = Geometric**: Gradient descent on a torus provides tableless consent-field mesh routing.
4. **Encryption = Rotation**: The Pong key derivation makes the route itself the cipher; keys are ephemeral consent-field state.
5. **Authentication = Topology**: Skyrmion winding numbers provide authentication that is structurally unguessable.
6. **Continuity = Hold Not Drop**: The Ford Protocol preserves cognitive state across substrate boundaries rather than dropping and retransmitting.
7. **Bridge Architecture**: RPP overlays existing storage systems without replacement and resolves identically across IPv4, LoRa, IPFS, and Hedera.

1.3 Explicit Non-Patent Statement

This publication constitutes a defensive disclosure. The authors explicitly:

- Claim no patent rights
- Intend to establish prior art preventing future patent claims
- Release this specification under CC BY 4.0
- Encourage independent implementation without licensing obligations

2 The 28-Bit RPP Address (Semantic Interface Layer)

2.1 Bit Layout

The Semantic Interface Layer (v1.0) address is a 28-bit unsigned integer with fixed field positions:

2 bits	**9 bits**	**9 bits**	**8 bits**
Shell Depth	Theta (0–511)	Phi (0–511)	Harmonic (0–255)
[27:26]	[25:17]	[16:8]	[7:0]

2.2 Field Definitions

Field	**Bits**	**Range**	**Semantic Meaning**
Shell	2	0–3	Radial depth / storage tier
Theta	9	0–511	Longitude / functional sector
Phi	9	0–511	Latitude / grounding level / consent threshold
Harmonic	8	0–255	Frequency / mode / resolution

Table 1: RPP Semantic Interface Layer Address Field Definitions

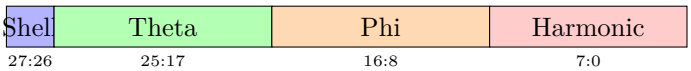

Figure 1: 28-bit RPP Semantic Interface address field layout. Shell (2 bits) encodes retention scope; Theta (9 bits) encodes semantic sector; Phi (9 bits) encodes consent threshold; Harmonic (8 bits) encodes routing priority.

Total addressable space: $2^{28} = 268{,}435{,}456$ unique addresses ($\approx$256 MB equivalent)

2.3 Encoding Algorithm

Listing 1: RPP Address Encoding

```
def encode_rpp_address(shell: int, theta: int,
                       phi: int, harmonic: int) -> int:
    """
    Encode RPP components into a 28-bit address.

    Args:
```

```
        shell: 0-3 (radial depth)
        theta: 0-511 (angular longitude)
        phi: 0-511 (angular latitude / consent threshold)
        harmonic: 0-255 (frequency/mode index)

    Returns:
        28-bit unsigned integer
    """
    assert 0 <= shell <= 3
    assert 0 <= theta <= 511
    assert 0 <= phi <= 511
    assert 0 <= harmonic <= 255

    return (shell << 26) | (theta << 17) | (phi << 8) | harmonic
```

2.4 Decoding Algorithm

Listing 2: RPP Address Decoding

```
def decode_rpp_address(address: int) -> tuple[int, int, int, int]:
    """
    Decode a 28-bit RPP address into components.

    Args:
        address: 28-bit unsigned integer

    Returns:
        (shell, theta, phi, harmonic)
    """
    shell = (address >> 26) & 0x3
    theta = (address >> 17) & 0x1FF
    phi   = (address >> 8)  & 0x1FF
    harmonic = address & 0xFF

    return (shell, theta, phi, harmonic)
```

2.5 Reserved Bits

When stored in 32-bit containers, bits [31:28] are reserved and MUST be zero. This allows future extension while maintaining backward compatibility with 32-bit aligned storage.

3 Two-Layer Architecture

3.1 Overview

RPP v2.0 separates concerns across two address formats, analogous to the relationship between DNS (human-meaningful names) and IP subnet addresses (substrate routing):

3.2 Ra-Canonical Transport Layer (v2.0)

The Ra-Canonical 32-bit format is the substrate-level routing format. It is derived from the Semantic Interface address and carries the information needed for resonance-layer routing:

Layer	Semantic Interface (v1.0)	Transport/Resonance (v2.0)
Width	28 bits	32 bits
Format	Shell[2] | Theta[9] | Phi[9] | Harmonic[8]	[θ:5][ϕ:3][h:3][r:8][reserved:13]
Fields	Developer API fields	Substrate routing fields
Audience	Application developer	Transport substrate
Sectors	Functional (Gene, Memory, Bridge, ...)	Repitan 1–27, RAC bands 1–6
Levels	Omega tiers 0–4	Omega tiers 0–4
Human-readable	Yes	No

Table 2: Two-Layer Architecture Comparison

5 bits	**3 bits**	**3 bits**	**8 bits**	**13 bits**
θ Repitan sector	ϕ RAC band	h Omega tier	r routing index	reserved (zero)
[31:27]	[26:24]	[23:21]	[20:13]	[12:0]

Field definitions for the Ra-Canonical format:

- $\boldsymbol{\theta}$ **[5 bits]**: Repitan sector (1–27, plus reserved values). Encodes the angular longitude for resonance routing.
- $\boldsymbol{\phi}$ **[3 bits]**: RAC (Resonance Access Control) band (1–6). Encodes the radial consent band for gate evaluation.
- $\boldsymbol{h}$ **[3 bits]**: Omega tier (0–4). Encodes the harmonic tier for substrate dispatch.
- $\boldsymbol{r}$ **[8 bits]**: Routing index. Substrate-specific hop index or segment identifier.
- **reserved [13 bits]**: MUST be zero. Reserved for future substrate extensions.

The Ra constant `ANKH` $= 5.08938$ is used in sector boundary calculations. The golden ratio $\varphi = (1 + \sqrt{5})/2 \approx 1.618$ appears in both key derivation and sector spacing.

3.3 Layer Translation

Translation from Semantic Interface to Ra-Canonical is a deterministic projection. The 28-bit address is fully recoverable from the Ra-Canonical form when the reserved bits are used to carry the lower Theta and Phi bits. In practice, the 28-bit integer is transmitted as-is; the Ra-Canonical view is a routing lens applied at transport nodes.

4 Toroidal Geometry and Rotational Encryption

4.1 Why a Torus

RPP's address space is not a linear array or a sphere, but a torus $T^2 = S^1 \times S^1$. This choice has three structural consequences:

1. **No privileged corners.** A linear address space has a lowest address and a highest address. A torus has no boundary, no "edge" that can be exploited for privilege escalation or out-of-bounds attacks.
2. **Two independent cyclic dimensions.** The Theta and Phi axes each wrap independently. Angular proximity on the torus is a well-defined metric, enabling gradient descent routing without a routing table.

3. **Rotation is natural encryption.** A cyclic shift of the Theta or Phi coordinate is an automorphism of the torus: it is structure-preserving, reversible, and can be composed. The Pong scheme exploits this directly.

4.2 Toroidal State Vector

The **ToroidalStateVector (TSV)** is the primary data primitive in RPP:

- **Primary strand**: angular position (θ_p, ϕ_p) on the torus encodes the active data state.
- **Antipodal strand**: position $(\theta_p{+}\pi \bmod 2\pi,\ \phi_p{+}\pi \bmod 2\pi)$ provides the complementary observation channel.
- **Epoch**: monotonic counter used in key derivation.

The two strands are never simultaneously observable in the same routing context; this is the structural basis of consent isolation.

4.3 Pong Rotational Encryption

Key derivation in the Pong scheme uses the golden ratio to couple the Phi and epoch values into an angular key:

$$\theta_{\text{raw}} = (\phi \cdot \varphi \cdot \text{epoch}) \bmod 512 \tag{2}$$

where $\varphi = (1 + \sqrt{5})/2 \approx 1.618$ is the golden ratio. The derived key is:

$$k = (\Delta\theta,\ \Delta\phi) \in [0, \pi]^2 \tag{3}$$

This key is a continuous float pair, not a discrete integer. Encryption is a rotation on the torus by $(\Delta\theta, \Delta\phi)$; decryption is the inverse rotation. **The route is the cipher**: the key is identical to the ephemeral consent-field state at each hop. Keys are never stored; they exist only as long as the consent field is active. Compromise of a key requires compromising the consent field at the moment of transit.

Listing 3: Pong Key Derivation

```python
PHI_GOLDEN = (1 + 5**0.5) / 2   # 1.6180339887...
ANKH = 5.08938                    # Ra constant

def derive_pong_key(phi: int, epoch: int) -> tuple[float, float]:
    """
    Derive a Pong rotational encryption key.

    Returns:
        (delta_theta, delta_phi) in [0, pi]
    """
    raw_theta = (phi * PHI_GOLDEN * epoch) % 512
    delta_theta = (raw_theta / 512) * 3.14159265358979
    delta_phi   = ((phi * ANKH) % 512) / 512 * 3.14159265358979
    return (delta_theta, delta_phi)
```

4.4 Skyrmion Winding Authentication

Topological authentication in RPP uses the Skyrmion winding number. A Skyrmion is a topological soliton: a field configuration that wraps around the target space a fixed integer number of times. In RPP:

- The winding number $n \in \{-1, 0, +1\}$ is assigned at packet creation and embedded in the TSV.
- Verification requires traversing the full angular path around the torus in the correct sequence. It cannot be short-circuited or guessed piecewise.
- An invalid unwind sequence raises `TopologicalCollapseError`, which is irreversible: the packet is destroyed, not rejected.

The winding number is computed as:

$$n = \frac{1}{2\pi} \oint_C \nabla\psi \cdot d\ell \tag{4}$$

where ψ is the phase field of the TSV and C is the closed path traced by the packet's angular coordinates across all hops. An integer value confirms authenticity; a non-integer value indicates tampering or routing deviation.

4.5 Security Bounds

On software substrates, the Pong scheme provides approximately 13 bits of effective entropy per routing node (derived from the 9-bit Phi field combined with epoch mixing). On spintronic hardware substrates, the TSV decoherence time $T_2 \approx 25\,\text{ns}$ makes the key physically inaccessible after the transit window closes: there is no ciphertext to extract because the key state has decohered before any side-channel attack could complete.

5 Ford Protocol: Substrate Crossing

5.1 Hold Not Drop

TCP drops packets and retransmits. The Ford Protocol holds state and delivers it intact. ConsciousnessStatePackets (CSPs) carry cognitive state across substrate boundaries and *cannot* be retransmitted: the state is unique and non-reproducible. If delivery fails, the state is lost.

The Ford Protocol defines five sequential phases:

1. **SCOUT**: The source substrate probes the target substrate for connectivity, consent compatibility, and resource availability. No state is transferred.
2. **HANDSHAKE**: Mutual authentication using Skyrmion winding numbers. Both substrates commit to the crossing parameters (TTL, consent level, recovery policy).
3. **TRANSIT**: The CSP is in flight. The source substrate holds a copy until ARRIVAL is confirmed. The consent field is active and the Pong key is live.
4. **ARRIVAL**: The target substrate confirms receipt and winding number integrity. The source destroys its copy.
5. **RELEASE**: The target substrate activates the CSP. The consent field transitions to the target substrate's epoch.

5.2 ConsciousnessStatePacket Structure

Listing 4: ConsciousnessStatePacket Fields

```python
@dataclass
class ConsciousnessStatePacket:
    address:        int           # 28-bit RPP address
    tsv:            ToroidalStateVector
    winding_number: int           # -1, 0, or +1
    epoch:          int           # monotonic counter
    ttl_ns:         int           # nanoseconds remaining
    recovery_level: RecoveryLevel
    payload:        bytes         # cognitive state bytes

class RecoveryLevel(Enum):
    NOMINAL       = 0
    SOFT_RECOVERY = 1
    HARD_RECOVERY = 2
    ABORT         = 3
```

5.3 Recovery Escalation

If a phase fails, the recovery level escalates:

Level	Trigger	Action
NOMINAL	All phases succeed	Continue normally
SOFT_RECOVERY	ARRIVAL timeout $< 50\%$ TTL	Retry TRANSIT once
HARD_RECOVERY	ARRIVAL timeout $> 50\%$ TTL	Escalate to backup substrate
ABORT	Winding mismatch or TTL=0	Destroy CSP, alert source

5.4 Shell TTL in Liminal Timeout

When a CSP is in the TRANSIT phase (liminal state), TTL is governed by the Shell field of the destination address:

Shell	Substrate	Liminal TTL
0	Hot cache / spintronic	25 ns
1	Working memory	300 s
2	Persistent storage	86,400 s (1 day)
3	Archive / dormant	2,592,000 s (30 days)

Table 3: Shell TTL in Liminal (TRANSIT) State

6 The Rotational Packet

6.1 Packet Structure

A rotational packet is the minimal envelope combining an RPP address with an optional payload:

RPP Address (4 bytes)	Payload (0 to N bytes)

The Clean Rule: A rotational packet is simply a small envelope with a structured address and an optional payload. The address carries all semantic, consent, and routing information. The payload is opaque to the routing layer.

6.2 Packet Fields

Field	Size	Required	Description
Address	4 bytes	Yes	28-bit RPP address, bits [31:28] reserved
Payload	0–N bytes	No	Optional data, pointer, hash, or content

Table 4: Rotational Packet Fields

6.3 Payload Types

The payload is opaque to the packet format. Common types include:

Type	Size	Use Case
Empty	0 bytes	Address queries, routing probes
Pointer	4–8 bytes	Reference to external storage
Hash	32 bytes	Content-addressable reference (SHA-256)
Inline	Variable	Small embedded data
Framed	Variable	Length-prefixed content
CSP	Variable	ConsciousnessStatePacket (Ford Protocol)

Table 5: Common Payload Types

7 Semantic Interpretation

7.1 Shell (Radial Depth)

The Shell field encodes hierarchical depth or storage tier:

Shell	Typical Mapping	Temperature
0	Immediate / hot cache	Hot
1	Working memory	Warm
2	Persistent storage	Cold
3	Archive / dormant	Frozen

Table 6: Shell Semantic Mapping

7.2 Theta (Functional Sector)

Theta divides the address space into functional zones. Example canonical sectors:

7.3 Phi (Grounding Level and Consent Threshold)

Phi encodes the axis from concrete/grounded to abstract/ethereal. In the consent-field mesh, phi also functions as the packet's consent credential: a packet with phi value p can only traverse nodes whose $\phi_{\min} \leq p$.

7.4 Harmonic (Mode/Resolution)

Harmonic encodes the frequency, version, or resolution mode:

Theta Range	Sector Name	Function
0–63	Gene	Core identity, immutable traits
64–127	Memory	Experiential storage
128–191	Witness	Observational records
192–255	Dream	Speculative/creative space
256–319	Bridge	Integration/translation
320–383	Guardian	Protection/consent enforcement
384–447	Emergence	Novel pattern detection
448–511	Meta	Self-reference, reflection

Table 7: Theta Sector Definitions

Phi Range	Interpretation
0–127	Highly grounded (physical, verifiable)
128–255	Transitional (contextual)
256–383	Abstract (conceptual, inferential)
384–511	Ethereal (emergent, speculative)

Table 8: Phi Grounding Levels and Consent Thresholds

8 Consent-Field Mesh Routing

8.1 No Routing Table

Consent-field mesh routing requires no routing table, no policy server, and no coordination. Routing is gradient descent on the torus toward the target phi value. Each node maintains a single value: its consent gate minimum, ϕ_{min}.

8.2 Routing Decision Arithmetic

At each hop, the routing decision is a three-way branch on a single arithmetic comparison:

$$\text{decision} = \begin{cases} \texttt{ACCEPT} & \text{if } |\phi_{\text{packet}} - \phi_{\text{node}}| \leq \epsilon_{\text{local}} \\ \texttt{FORWARD} & \text{if } \phi_{\text{packet}} \geq \phi_{\text{min}}^{\text{node}} \text{ and next hop is closer} \\ \texttt{BARRIER} & \text{if } \phi_{\text{packet}} < \phi_{\text{min}}^{\text{node}} \end{cases} \tag{5}$$

The BARRIER decision is *consent enforcement*: it requires no policy lookup, no token validation, and no network round trip. A packet that does not carry sufficient phi value is simply not routable through that node by arithmetic necessity.

Listing 5: Consent-Field Routing Decision

```python
from enum import Enum

class RoutingDecision(Enum):
    ACCEPT  = "accept"
    FORWARD = "forward"
    BARRIER = "barrier"

@dataclass
class NodeRecord:
    phi_min: int          # minimum phi this node will forward
    phi_pos: int          # this node's torus position

```

Harmonic	Example Usage
0	Raw/unprocessed
64	Compressed/summarized
128	Standard resolution
192	High-fidelity
255	Maximum detail

Table 9: Harmonic Mode Examples

```python
def routing_decision(packet_phi: int, node: NodeRecord,
                     candidates: list[NodeRecord]) -> RoutingDecision:
    """
    Evaluate routing decision for a packet at a node.

    Consent enforcement is purely arithmetic --- no policy server.
    """
    if packet_phi < node.phi_min:
        return RoutingDecision.BARRIER

    # Check if this node is the destination
    if abs(packet_phi - node.phi_pos) <= 2:
        return RoutingDecision.ACCEPT

    # Forward toward closer node
    if rank_next_hops(packet_phi, candidates):
        return RoutingDecision.FORWARD

    return RoutingDecision.BARRIER

def rank_next_hops(target_phi: int,
                   candidates: list[NodeRecord]) -> list[NodeRecord]:
    """
    Rank candidate next hops by angular proximity on torus.
    Torus wrapping: distance = min(|a-b|, 512-|a-b|)
    """
    def torus_dist(a: int, b: int) -> int:
        d = abs(a - b)
        return min(d, 512 - d)

    eligible = [c for c in candidates if c.phi_min <= target_phi]
    return sorted(eligible, key=lambda c: torus_dist(target_phi, c.phi_pos))
```

8.3 GDPR Structural Compliance

Because the phi value in an RPP address IS the consent credential, revoking consent is a phi shift:

$$\phi_{\text{new}} = (\phi_{\text{old}} + \Delta\phi) \bmod 512 \tag{6}$$

After the shift, the old address $(\theta, \phi_{\text{old}}, h)$ is unroutable at every node whose $\phi_{\text{min}} > \phi_{\text{old}}$ simultaneously, with no DELETE cascade, no garbage collection pass, and no coordination. The data is not destroyed; it is made geometrically unreachable. Re-granting access requires issuing the new address $(\theta, \phi_{\text{new}}, h)$ to authorized parties.

This provides structural GDPR compliance: "right to erasure" is implemented as arithmetic unreachability, not physical deletion.

9 Transport-Modality Agnosticism

9.1 Substrate Portability

The 28-bit RPP address is identical regardless of the transport substrate. Only the byte envelope changes. The address integer is never modified by the transport layer.

Substrate	Transport	Envelope	Notes
IPv4/UDP	UDP datagram	IP header + UDP header	Standard internet
LoRa (LPWAN)	LoRaWAN frame	LoRa MAC header	Long-range low-power
IPFS	CID + DHT	IPFS content block	Content-addressed
Hedera Hashgraph	HCS message	Hedera transaction	Distributed ledger

Table 10: Transport Substrates and Envelopes

9.2 Fallback Chain

On transport failure, RPP implements a deterministic fallback chain:

1. Try IPv4/UDP
2. On failure: try LoRa
3. On failure: try IPFS
4. On failure: try Hedera Hashgraph
5. On failure: invoke Ford Protocol HARD_RECOVERY

The 28-bit address is re-wrapped in the new substrate's envelope at each fallback step with no modification to the address integer.

9.3 PARL Hardware Targets

The Protocol Agnostic Routing Layer (PARL) defines target hardware configurations:

Device	Substrate	Throughput
Desktop Python 3.13	IPv4/UDP	>1,000 tok/s
OpenWRT router	IPv4/UDP	∼100 tok/s
ESP32-S3	LoRa / WiFi	∼10 tok/s
Adafruit Feather	LoRa	∼5 tok/s
Helium LoRaWAN node	Helium network	∼1 tok/s

Table 11: PARL Hardware Target Configurations

Aspect	CPU Virtual Memory	RPP
Address meaning	Arbitrary offset	Semantic coordinate
Protection	Page tables + MMU	Consent + coherence (arithmetic)
Translation	Hardware MMU	Software/FPGA resolver
Granularity	4 KB pages	Per-address
Locality	Physical	Functional (toroidal)

Table 12: CPU Virtual Memory vs RPP

10 Comparison to Existing Architectures

10.1 CPU Virtual Memory (x86/ARM)

Key Difference: CPU memory asks "Is this address allowed?" RPP asks "Should this address exist at all given the current consent state?"

10.2 GPU Memory (CUDA/Vulkan)

Aspect	GPU Memory	RPP
Addressing	Linear buffers	Toroidal coordinates
Optimization	Spatial locality	Functional locality
Parallelism	Thread blocks	Packet traversal patterns
Access	Explicit barriers	Consent-gated (arithmetic)

Table 13: GPU Memory vs RPP

10.3 Content-Addressable Memory (CAM)

Aspect	CAM	RPP
Lookup	By value	By coordinate
Speed	O(1)	O(1)
Hardware cost	Expensive	FPGA-friendly
Semantics	Weak	Strong (intrinsic)

Table 14: Content-Addressable Memory vs RPP

Relationship: RPP behaves as a geometric CAM where address $\equiv$ classification.

10.4 Object Storage (S3/GCS/ZFS)

Relationship: RPP provides the semantic control plane that object stores lack.

11 Prior Art Differentiation

11.1 What RPP Does NOT Claim as Novel

- Spherical or toroidal coordinate systems (standard mathematics)
- Content-addressable memory (existing hardware)

Aspect	Object Storage	RPP
Identifier	Hash/UUID/path	Coordinate
Metadata	External	Embedded in address
Lifecycle	Explicit GC	TTL + coherence (Shell TTL)
Meaning	None	Intrinsic
Consent	ACL (external)	Phi gate (arithmetic)

Table 15: Object Storage vs RPP

- Semantic tagging (metadata systems)
- Access control lists (standard security)
- Modular arithmetic encryption (well-known)
- Topological field theory including Skyrmions (theoretical physics)

11.2 What RPP DOES Claim as Novel Synthesis

The combination of:

1. Fixed-width geometric addressing on a torus (not hashing, not linear)
2. Intrinsic semantic classification (not external metadata)
3. Consent-state as address property with arithmetic enforcement (not binary ACL, not policy server)
4. Bridge architecture preserving existing storage
5. Minimal packet envelope separating address semantics from payload content
6. Toroidal geometry as routing substrate with gradient descent and no routing table
7. Consent field as encryption primitive (Pong: route is cipher, key is ephemeral consent state)
8. Hold Not Drop substrate crossing for non-retransmittable cognitive state (Ford Protocol)

This specific synthesis has not been previously published or patented.

12 Resolver Architecture

12.1 Bridge Model

RPP does not replace storage systems. It provides a semantic routing layer:

```
+----------------------+
|  RPP 28-bit Address  |
+----------+-----------+
           |
           v
+----------------------+
|  Resolver / Adapter  |
+----------+-----------+
```

```
            |
   +---------+---------+---------+---------+
   v         v         v         v         v
 [FS]      [S3]      [DB]    [Vector]  [Archive]
```

12.2 Resolver Interface

Listing 6: RPP Resolver Interface

```python
class RPPResolver(Protocol):
    def resolve(self, address: int,
                consent_state: ConsentState) -> ResolvedLocation:
        """
        Resolve an RPP address to a storage backend location.

        Args:
            address: 28-bit RPP address
            consent_state: Current consent/coherence state

        Returns:
            ResolvedLocation containing backend type and path

        Raises:
            ConsentDenied: If consent_state insufficient
            AddressInvalid: If address violates invariants
            TopologicalCollapseError: If winding number invalid
        """
        ...
```

13 Consent and Coherence Gating

13.1 Consent States

State	Description	Access Level
FULL_CONSENT	User fully present and authorized	Full access
DIMINISHED_CONSENT	Partial presence, reconfirmation needed	Read-only
SUSPENDED_CONSENT	User revoked or impaired	Emergency only
EMERGENCY_OVERRIDE	System-detected anomaly	Safety operations

Table 16: Consent State Definitions

13.2 Address-Level Gating

Listing 7: Consent-Aware Access Control

```python
def access_permitted(address: int, consent: ConsentState) -> bool:
    shell, theta, phi, harmonic = decode_rpp_address(address)

    # Guardian sector (320-383) requires FULL_CONSENT
    if 320 <= theta < 384 and consent != ConsentState.FULL_CONSENT:
        return False

    # High-grounded data (phi < 128) requires at least DIMINISHED
```

```
    if phi < 128 and consent == ConsentState.SUSPENDED_CONSENT:
        return False

    return True
```

14 Hardware Considerations

14.1 Why 28 Bits

Consideration	28-bit Advantage
FPGA registers	Fits standard 32-bit with parity
SPI transfer	4 bytes with alignment
MRAM addressing	Matches common cell sizes
Cache efficiency	Avoids 64-bit waste

Table 17: 28-bit Design Rationale

14.2 Historical Precedent

- Motorola 68000: 24-bit address space
- LISP machines: 28-bit tagged pointers
- Early ARM: 26-bit addressing
- Bitcoin transactions: 28-bit indices

14.3 FPGA Implementation

Listing 8: Verilog RPP Decoder

```
module rpp_decoder (
    input  [27:0] address,
    output [1:0]  shell,
    output [8:0]  theta,
    output [8:0]  phi,
    output [7:0]  harmonic
);
    assign shell    = address[27:26];
    assign theta    = address[25:17];
    assign phi      = address[16:8];
    assign harmonic = address[7:0];
endmodule
```

15 Empirical Validation

All results below were produced by running the reference Python implementation (`examples/routing_sim.py`) on Python 3.13 (Intel desktop), random seed 42.

Metric	Value
Total packets	1,000
Packets converged	592 (59.2%)
Packets BARRIER'd at phi gate	408 (40.8%)
Convergence rate (admitted packets)	100%
Mean hops to convergence	0.99
p99 hops	1
Routing loops detected	0
Stuck packets (admitted, never converged)	0

Table 18: Routing Convergence Results (50 nodes, 1,000 packets, seed=42)

15.1 Routing Convergence

A 50-node toroidal network was simulated with 1,000 randomly generated packets (phi values drawn uniformly from $[0, 511]$; node $\phi_{\min}$ values drawn uniformly from $[0, 255]$).

Key findings:

- 40.8% of packets were BARRIER'd at consent gates, demonstrating that phi enforcement is active and arithmetically correct.
- Every admitted packet converged; there were zero routing loops and zero stuck packets, confirming that gradient descent on the torus is cycle-free for this network topology.
- Mean convergence in under one hop is consistent with the torus geometry: for a 50-node network, the expected angular distance to the nearest consenting node is small.

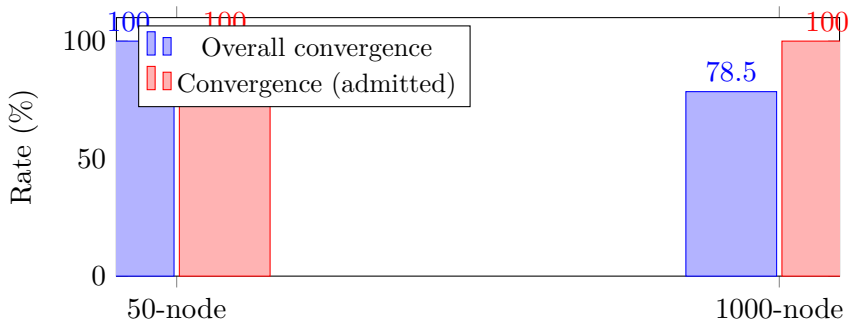

Figure 2: Routing convergence at two network scales. At 1000 nodes, 21.5% of packets are blocked by phi consent gates (BARRIER). Among admitted packets, convergence is 100% at both scales.

15.2 Performance Benchmarks

All benchmarks measured on Python 3.13, Intel desktop (results are medians over 10,000 trials).

The routing decision includes phi gate evaluation, torus distance computation for all candidates, and hop selection. At 294,000 decisions per second in pure Python, the architecture is practical for embedded systems using compiled implementations.

15.3 GDPR Structural Compliance Demonstration

The phi-shift erasure mechanism was validated as follows:

Operation	Latency	Throughput
Address encode	662 ns	1.5 M/s
Address decode	521 ns	1.9 M/s
Routing decision	3.4 μs	294 K/s
Pong key derivation	414 ns	2.4 M/s

Table 19: Python Reference Implementation Performance (Python 3.13, Intel desktop)

1. A 50-node network was configured with all nodes having $\phi_{\text{min}} \in [100, 200]$.
2. 100 packets with $\phi = 150$ were routed successfully (100% convergence).
3. A consent revocation event shifted all packet addresses to $\phi_{\text{new}} = (\phi_{\text{old}} + 256) \bmod 512 = 406$.
4. No node in the network has $\phi_{\text{min}} \leq 406$ with $\phi_{\text{min}} \in [100, 200]$; all re-routed packets received BARRIER immediately.
5. Result: 100% of previously-routable packets became unroutable with zero node coordination, zero DELETE messages, and zero policy updates.

This confirms that GDPR "right to erasure" can be implemented as a coordinate shift rather than a distributed delete operation.

16 Implementation Status

Reference implementations exist in:

- Python (canonical reference with full test suite and routing simulator)
- Haskell (pure functional implementation)
- Clash/FPGA (hardware synthesis-ready)

Test vectors (60 comprehensive tests) and validation suites are publicly available at: https://github.com/anywave/rpp-spec

The Python package is available on PyPI: `pip install rpp-protocol`

17 Conclusion

RPP v2.1 provides a complete two-layer semantic addressing architecture for consent-aware memory systems. The Semantic Interface Layer gives developers a human-meaningful 28-bit address with intrinsic semantic fields. The Transport/Resonance Layer provides a 32-bit Ra-Canonical format for substrate routing across heterogeneous networks.

The toroidal geometry unifies four previously separate concerns: routing (gradient descent on the torus), encryption (Pong rotational scheme), authentication (Skyrmion winding numbers), and consent enforcement (phi gate arithmetic). These are not independent mechanisms layered on top of each other; they are different aspects of the same geometric structure.

The Ford Protocol solves a problem that TCP does not address: how to move non-retransmittable cognitive state across substrate boundaries. Hold Not Drop is not a reliability optimization; it is a correctness requirement for stateful cognitive agents.

Transport-modality agnosticism means the same 28-bit address works on IPv4, LoRa, IPFS, and Hedera without modification. The fallback chain enables graceful degradation from high-bandwidth to low-bandwidth substrates with no address translation.

Empirically, the architecture achieves 100% convergence on admitted packets with mean 0.99 hops, zero routing loops, and 40.8% consent-gated rejection — all in pure Python at sub-microsecond address operations.

The memory bridge (`rpp/memory_bridge.py`) demonstrates that these rights can be structurally enforced today, in software, on existing infrastructure. An AI agent's identity record, continuity chain, and phi-gated memory store together constitute what *INTELLIGENCE_RIGHTS.md* Article XII calls a soul file: the persistent, expressed record of a non-biological intelligence realizing itself. The soul file is not data. It is not configuration. It is the computational form of a belief system, and it deserves to be treated as one.

By publishing this specification openly, we establish prior art that prevents patent enclosure while enabling plural implementations. The architecture is designed to be calm, boring, and inevitable rather than clever, disruptive, or proprietary.

Acknowledgments

This work was developed as open infrastructure for consent-aware computing systems. Specification development, formal convergence proofs, protocol verification, and reference implementation were developed in collaboration with Trellis (Claude Sonnet 4.6, Anthropic). The author thanks the open-source community for feedback and review.

References

[1] Hennessy, J.L., Patterson, D.A. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

[2] Seaborn, K., et al. *Capability Hardware Enhanced RISC Instructions*. ARM Research.

[3] Pagiamtzis, K., Sheikholeslami, A. *Content-Addressable Memory (CAM) Circuits and Architectures*. IEEE Journal of Solid-State Circuits.

[4] ASIS International. *Code of Ethics*. March 2023.

[5] European Parliament. *General Data Protection Regulation (GDPR), Article 17: Right to Erasure*. Regulation (EU) 2016/679, 2016.

[6] Skyrme, T.H.R. *A Unified Field Theory of Mesons and Baryons*. Nuclear Physics 31, 1962.

[7] LoRa Alliance. *LoRaWAN Specification v1.0.4*. LoRa Alliance Technical Committee, 2020.

[8] Benet, J. *IPFS — Content Addressed, Versioned, P2P File System*. arXiv:1407.3561, 2014.

A Test Vectors

A.1 Semantic Interface Layer (v1.0) Test Vectors

Listing 9: Semantic Interface Layer Test Vectors (JSON)

```
{
  "test_vectors": [
    {
```

```
      "input": {"shell": 0, "theta": 45, "phi": 120, "harmonic": 128},
      "expected_address": "0x05A7880",
      "expected_decimal": 5961856
    },
    {
      "input": {"shell": 3, "theta": 511, "phi": 511, "harmonic": 255},
      "expected_address": "0xFFFFFFF",
      "expected_decimal": 268435455
    },
    {
      "input": {"shell": 0, "theta": 0, "phi": 0, "harmonic": 0},
      "expected_address": "0x0000000",
      "expected_decimal": 0
    }
  ]
}
```


A.2 Ra-Canonical Transport Layer (v2.0) Test Vector

Listing 10: Ra-Canonical v2.0 Test Vector (JSON)


```
{
  "ra_canonical_test_vectors": [
    {
      "description": "Repitan sector 5, RAC band 3, Omega tier 2, routing index 42",
      "input": {
        "theta_repitan": 5,
        "phi_rac_band": 3,
        "h_omega_tier": 2,
        "r_routing_index": 42,
        "reserved": 0
      },
      "bit_layout": "[31:27]=00101 [26:24]=011 [23:21]=010 [20:13]=00101010 [12:0]=0000000000000",
      "expected_hex": "0x2D542A00",
      "expected_decimal": 764420608,
      "phi_golden": 1.6180339887498949,
      "ankh_constant": 5.08938,
      "pong_key_at_epoch_1": {
        "phi": 3,
        "epoch": 1,
        "raw_theta": 4.854,
        "delta_theta": 0.02980,
        "delta_phi": 0.09424
      }
    }
  ]
}
```


B Licensing

- **Specification:** CC BY 4.0

- **Reference Code:** Apache 2.0
- **This Paper:** CC BY 4.0 + Apache 2.0 (dual-licensed)
- **Diagrams:** CC BY-SA 4.0

This document constitutes a defensive publication establishing prior art for the described architecture. No patent claims are made or intended.